

Progress Report 1

Natalia Leyenda Lodaes, Miguel Montes Lorenzo,
Sofía Pedrós Tobaruela, Manuel Rodríguez Villegas

April 2024

1 Introduction

During this week, we have progressed with the basic models (Naive Bayes) as well as some recursive models. Additionally, we have almost completed a first Embedding model using dependency parsing.

2 Models

2.1 Recursive Models

We have understood how a recursive model work and we have successfully implemented a basic RNN and a RNTN. We have used the same class with a boolean attribute, as the RNN is the same as the RNTN except for Parameter "V", that in the RNN is zero and in the RNTN is another trainable parameter.

A recursive model (also referred as RvNN, RNNN, RecNN or even RNN) is a type of neural network architecture that is specially designed to process hierarchical structures and capture dependencies within recursively structured data. This model allows us to obtain structured predictions from structured inputs applying weights to the inputs in a recursive way. The structure of the model is a tree that goes from the leaves to the root. These trees have a fixed number of children per node, to apply the same weights in each step. For example, in a binary tree with a simple model, where p is the parent, $c1$ and $c2$ the children and a is an activation function:

$$p = a \left(W \begin{pmatrix} c1 \\ c2 \end{pmatrix} + b \right) \quad (1)$$

Our model will start with the embeddings for each word and, with the tree structure provided in the dataset, it will start computing the "embeddings" of each constituent. The embeddings (d-dimensional) can be a trainable parameter (in which case it will be initialized from a uniform distribution), or they are going to be pretrained (when we use the dependency parsing). With the final "sentence embedding" the model will classify using a SoftMax between 5 classes. In that case, the label for the sentence is:

$$y^a = softmax(W_s * a) \quad W_s \in R^{5 \times d} \quad a \in R^d \quad (2)$$

The difference between the different recursive models resides in how to compute the parent from its children.

Recursive models can be used for sentiment analysis as it allows us to see the syntactic structure of the phrase and how does each element influence in the meaning. In that sense, a recursive model saves the complete sentence representation, so constituency parsing is completely done by a recursive model.

For this part of the project, we have understood how does the forward and backward through structure works, following the explanation for the Stanford paper [2] and lecture on recursive models [3]. Then, we have studied how to create a class Tree, with the SST dataset. With that class, we've created the basic structure of a recursive model that uses that Tree to propagate the operations from the leaves to the root. Finally, we've completed the forward using the tree propagation method and the backward is completely implemented by Pytorch. To confirm it works, we have looked at the derivatives from W, during a complete train execution. We have checked the results are coherent with the expected values (not NaN values). Additionally, visualizing the loss function, it appears to descend correctly, but it gets stuck in 0.1. It is still to be determined if the backward is correct.

2.1.1 Recursive Neural Model

This is the simplest model, each parent is computed multiplying their children for a matrix of weights W and then applying an activation function. A bias can also be added.

$$p_i = \text{softmax}(W_s * a) \quad W \in R^{dx2d} \quad p_i \in R^d \quad (3)$$

2.1.2 Recursive Neural Tensor Network

This model adds a new parameter: V . The idea behind this model is to represent more relationships between the words, using the idea of MV-RNN (a recursive model that represents each node as a vector and a matrix) but having a fixed number of parameters. The "V" matrix allows the model to store a more direct relationship between input nodes so it has more interactions between the inputs. The new equation to compute parent nodes would be:

$$p_i = a \left(\begin{pmatrix} c1 \\ c2 \end{pmatrix}^T V^{[1:d]} \begin{pmatrix} c1 \\ c2 \end{pmatrix} + W \begin{pmatrix} c1 \\ c2 \end{pmatrix} \right) \quad \text{where } V^{[1:d]} \in R^{2dx2dx2d} \quad p_i \in R^d \quad W \in R^{dx2d} \quad c_i \in R^d \quad (4)$$

2.2 Embeddings

To add an extra syntax source to the project, we have decided to use our own embeddings combining the SGNS algorithm with information from dependency parsing trees. We have done so by adding to the context words of SGNS the words that are directly connected in the dependency graph, both head and dependents, as explained in [1]. This model has been trained using sentences from the Stanford dataset and movies reviews from the IMBD dataset. To measure their performance we will compare them to the BERT embeddings, which also add contextual information.

2.3 Naive Bayes

Since one of the objectives of the project is to compare the performance of traditionally used classification models with those that incorporate syntax-aware classification, we have applied the Naive Bayes model to the Stanford dataset, which is the one that will presumably be used to carry out the benchmarking. We are going to use Naive Bayes with BoW and also with word embeddings.

3 Plan

For the future of the recursive models, we have to train them and finetune the hyperparameters and confirm if it is training correctly. Additionally we have to test the models with both types of embeddings: pretrained and trainable to compare the results. If there is time left, we will also try to implement the MV-RNN model. Furthermore, a traditional RNN will be implemented for comparison purposes and we will finish the Naive Bayes implementation.

References

- [1] Omer Levy. *Dependency-Based Word Embeddings*. 2014. URL: <https://aclanthology.org/P14-2050.pdf>.
- [2] Richard Socher. *Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank*. 2013. URL: https://nlp.stanford.edu/~socherr/EMNLP2013_RNTN.pdf.
- [3] Stanford University School of Engineering. *Lecture 14: Tree Recursive Neural Networks and Constituency Parsing*. 2017. URL: <https://www.youtube.com/watch?v=RfwgqPkWZ1w>.